

RTL Design Sherpa

RAPIDS Beats DMA Engine Performance Characterization Report

July 15, 2026

Table of Contents

Readme....5

RAPIDS Beats DMA — Performance Characterization (Genesys 2, 8 channels)....6

Characterization knobs....10

1. Headline....11

2. How it is measured (the observation hooks)....13

3. Single-axis sweep — transfer size....14

3.1 Channel count — per-channel independence....16

4. What we learned....18

5. Between-run state (harness workaround + open RTL defect)....19

6. Other limitations....20

7. The STREAM knob set, measured by the interface observers (v1.2)....21

7.0 The regression this campaign found first....21

7.1 Headline: line rate on all four interfaces....22

7.2 Transfer size (phase C): the amortization knee....23

7.3 Descriptors x channels (phase A): the STREAM matrix....25

7.4 Burst length (phase B): the transaction-size knee....26

7.5 Memory latency (phase E): the in-flight window....27

7.5c Buffer depth in the sequential knee (8-cycle steps, 4 KB vs 16 KB)....30

7.6 What the observers add over the bare meters....33

7.7 One channel under latency: burst length is the write side's lever, SRAM depth the read side's bound....33

Appendix: data files & reproduce....36

List of Figures

No figures in this document.

List of Tables

Table 1: Characterization knobs — the sweep axes.....10
Table 2: Headline — 8-channel line-rate at 128 KB/channel (Genesys 2, 256-bit build).....11
Table 3: Table — engaged utilization (%) and effective bandwidth (GB/s) vs transfer size (8 ch, 256-bit build).....15
Table 4: Figure — engaged utilization vs active channel count (128 KB/ch).....17
Table 5: Table — the wedge is stale scheduler state, not an in-flight fabric cycle..19
Table 6: Table 7.1 – observer engaged utilization vs active channels (phase D, 256-bit build).....22
Table 7: Table 7.2 – size sweep at 8 channels (256-bit build); latencies are observer-histogram means in aclk cycles.....23
Table 8: Table 7.3 – AXI4-wr (sink) / AXIS-out (source) utilization over the descriptor x channel matrix (256-bit build; every cell equals the 512-bit build's).25
Table 9: Table 7.4 – burst-length sweep (256-bit build); latencies in aclk cycles….26
Table 10: Table 7.5 – latency sweep, sequential channel schedule (v2.0, 256-bit build, PIPELINE = 1, HIST_MAX_OUTSTANDING = 32, no sample loss on any row); the latency columns are log2-histogram means, so they sit on bin midpoints.....28
Table 11: Table 7.5b – latency sweep, interleaved channel schedule (v2.0, same bitstream, PIPELINE = 1, HIST_MAX_OUTSTANDING = 32, no sample loss on any row; 12/12 CRC-verified). The source columns are the same run and match Table 7.5, as they must: the schedule only changes the sink's stimulus. With every channel holding data the sink ingress reads 100.0 % here: the 82-cycle fill stall of the sequential schedule is one channel filling alone.....29
Table 12: Table 7.5c – fine-step latency sweep, 4 KB (128-beat) vs 16 KB (512-beat) buffers, 256-bit build, sequential schedule (genesys_dw256_fine_{4kb,16kb}_{8ch,1ch}.json, 64/64 CRC-verified).....30
Table 13: Table 7.7 – one channel, AXI4-wr / AXI4-rd utilization (%) vs burst length and injected latency, on both design points (genesys_one_channel_xfer_latency.json, genesys_dw256_one_channel_xfer_latency.json).....34

Readme

Generated: 2026-10-20

RAPIDS Beats DMA — Performance Characterization (Genesys 2, 8 channels)

v2.2 (2026-09-29). rapids BUG-009 root-caused and fixed, and section 7.7 corrected. Three mechanisms, each caught on the ILA. The “burst equal to the SRAM depth” wedge was an 8-bit `AxLEN + 1` wrapping to 0 at AxLEN 255 in both engines: a 256-beat burst reserved nothing, and the write side’s drain count on that channel never recovered (`CHANNEL_RESET` does not reach it), so every later run with a burst of 120 beats or more wedged – which is where v2.1’s “128 beats on the 4 KB build” rows came from. Underneath it, the sink ingress allocated SRAM only in whole 16-beat segments, so any run that ended mid-segment left the last few slots unreachable and a burst that needs every slot waited forever; and the partial allocation that fixes that had to respect the three-cycle lag of the space view, or it over-allocated and dropped beats. The engines clamp the burst to the buffer (AxLEN 255 runs as 128-beat bursts on this build), the ingress allocates the remainder when less than a segment is free and settles three cycles after every allocation, Table 7.7 and the channel x size matrix are re-measured on the fixed bitstream (no other cell moved by more than 0.5 pt), and every results row carries `xfer_axlen_effective`. Build: WNS +0.286 ns.

v2.1 (2026-09-29). Section 7.5c added: the latency sweep in 8-cycle steps on the 4 KB build and on a 16 KB-per-channel variant of the same 256-bit design (512 beats; `RAPIDS_SRAM_DEPTH=512`, WNS +0.651 ns). Sean’s point, measured: at 8 channels the deeper buffer moves the sink knee out (85.9 % vs 55.2 % at 128 cycles) because the generator can run ahead into the next channels’ buffers while the current one drains; at one channel the two buffers coincide to the decimal, since there is no next channel to run into and the window is the outstanding cap. v2.0’s 7.5 text said the buffer was not in the knee; that was wrong for the 8-channel sequential rows and is corrected below. The design point stays 4 KB.

v2.0 (2026-09-29). New design point. Sean: AXI4 read/write engines at **256 bits**, the AXIS links halved to 256 with them (RAPIDS has one `DATA_WIDTH` for both), and **4 KB of SRAM per channel** (128 beats x 32 B; the v1.x builds had 16 KB = 256 x 64 B). Line rate is therefore **3.20 GB/s** per direction (32 B x 100 MHz) instead of 6.40. Every table and figure in sections 1, 3 and 7 is re-measured on one observers bitstream of this

build (post-route WNS +0.608 ns, 75679 LUTs, 44 BRAM tiles, against +0.351 / 80040 / 65.5 at 512 bits); the bitstream reports its geometry in the harness `BUILD` register and the host takes its byte and bandwidth math from there. The result, cell for cell: the same utilization at the same beat count as the 512-bit build, so the engine's per-beat behaviour is width-independent and every GB/s figure simply halves. Two things the smaller buffer changed: the sink ingress now pays a fixed 82-cycle fill stall per run (AXIS-in reads 96.1 % at 8 KB and 99.7 % at 128 KB per channel instead of 100 %), and the single-channel write window in the latency sweep shrank from ~93 to ~80 beats. Section 7.7 (new) is the single-channel burst-length sweep behind the knob answer. The v1.5 PDF stands as the record of the 512-bit build; its data files stay in the appendix.

v1.5 (2026-09-28). The 7.5 latency sweep re-measured with the harness generator's new interleaved-channel schedule (rapids TASK-018: `GEN_MODE. INTERLEAVE`, `run_characterization.py --interleave`), which round-robins the eight channels one beat at a time so every sink channel holds data at once. The sink column therefore now measures the DUT's aggregate write window instead of one channel's: Table 7.5b shows the sink write path holding 99.9 % on every row to 512 cycles of injected latency, with no knee inside the sweep. Table 7.5 (sequential schedule, one channel's window) stands, and re-measured cell for cell on the v1.5 bitstream (observers build with `GEN_MODE`, post-route WNS +0.245 ns). No other section changed.

v1.4 (2026-09-28). Sections 1, 3 and 7 re-measured on a bitstream built from the current RTL (rapids TASK-015 AXIS monitor-lite skids on both network ports, BUG-004..007 engine fixes, PIPELINE = 1). Two things changed the numbers: the sink-ingress meter window now brackets first beat to last beat (rapids ISSUE-001), so the AXIS-in column no longer charges the ~200-cycle launch cost to ingress and reads 97-100 % from 4 beats up; and the write engine now runs with PIPELINE = 1. The first bitstream with the BUG-005 fix at the old PIPELINE = 0 measured 49.9 % AXI4-wr at 4096 beats/channel (59.5 % at 1024): the pre-fix engine had reached line rate only because it ran two bursts in flight per channel by accident, and one-in-flight cannot cover the write response round trip at 8-beat bursts. PIPELINE = 1 (up to AW_MAX_OUTSTANDING = 8 per channel) is the design point from here on and is what this version

reports. Rapids ISSUE-004 (one row reading zero ingress starvation) closed on this re-measurement plus an ILA trace of that row: every row now shows the same single trailing starvation cycle. The 7.5 sink column measures one channel's window because the generator feeds channels one at a time (rapids ISSUE-006, closed the same day; an interleaved mode is rapids TASK-018).

v1.3 (2026-09-27). Latency sweep (7.5) re-measured on a bitstream whose AXI observer keeps 32 timestamps per channel instead of 8: no histogram sample loss on any row, latency columns replaced, caveat retired (rapids TASK-012). Every utilization and bandwidth number is unchanged from v1.2.

v1.2 (2026-09-27). Section 7 adds the STREAM five-knob characterization measured by the shared interface observers (`axi4_intf_master_observer` on the AXI4 masters, `axis4_intf_observer` on both AXIS links) instead of the harness's bare meters, on a bitstream that also fixes a source-path regression the campaign found (rapids BUG-003 / stream BUG-011). Sections 1-6 are the v1.1 bare-meter record and stand as measured.

Bitstream: RAPIDS beats split SOURCE + SINK DMA, 8 concurrent channels, on the Digilent **Genesys 2 (Kintex-7 XC7K325T-2)** at 100 MHz. The AXI4 and AXIS4 datapaths are **256 bits wide (32 B/beat)** since v2.0 (512 bits / 64 B through v1.5), so the one-direction line rate is **3.20 GB/s** (`32 B × 100 MHz`; 6.40 at 512 bits). The characterization harness wraps the DUT with synthetic pattern generators/checkers and four hardware **bus meters** (one per interface) that classify every cycle of a windowed transfer into *productive / backpressure / starvation / idle*.

Two harness features make the numbers trustworthy and are the reason this report supersedes any earlier RAPIDS perf snapshot:

1. **Atomic launch (stage-all-then-GO).** Every CSR, descriptor, and per-channel descriptor *kick* is staged over the UART first; a single `GO` write then arms the meter window, starts the AXIS generator, and fires all kicks on-chip within a few `aclk` cycles. No UART latency contaminates the measured window.
2. **Deterministic window close.** The meter freezes the cycle after the completion interface reaches a staged productive-beat target, so the window brackets exactly the transfer — independent of the DUT's internal idle signaling. Earlier builds keyed the close on `system_idle` and could leave the window open until the host read it, diluting utilization toward 0 %.

Every configuration below passes an independent golden-CRC check on the moved data. **Source of truth:** the on-chip bus meters, read over CSR.

> **Metric note.** The headline efficiency is **engaged utilization** — `productive / (productive + backpressure + starvation)`, i.e. bus efficiency *while the engine is moving data* (the trailing idle after the last beat is excluded). Effective bandwidth is `engaged_util × line rate` (3.20 GB/s at 32 B/beat; the v1.x tables were at 6.40). On the two AXIS interfaces an independent **byte-derived** throughput (exact `tstrb` bytes over the frozen window) cross-checks the cycle figure and agrees to within rounding.

Characterization knobs

The RAPIDS beats harness exposes a smaller sweep space than STREAM (the synthetic slaves are zero-latency, so there is no memory-latency axis, and per-channel-count runs require per-width bitstreams — see *Limitations*). The axes exercised here:

Characterization knobs — the sweep axes

#	Knob	What it is	Set by	Values in this report
1	**Transfer size** (beats/channel)	beats moved per channel per descriptor — the amortization axis	descriptor `length` / `GEN_NBEATS`	**128 B (4 beats) … 128 KB (4096 beats)**
2	**Path**	SOURCE (mem-read → AXIS-out) vs SINK (AXIS-in → mem-write)	which half is kicked	both, every config
3	**Interface**	the four monitored buses	—	AXIS-in, AXI4-wr, AXI4-rd, AXIS-out
4	**Channels**	concurrent DMA channels	build generic	**8** (full width)

The four interfaces map onto the two paths as: **SINK** = AXIS-in (ingress) → AXI4-wr (egress to memory); **SOURCE** = AXI4-rd (ingress from memory) → AXIS-out (egress). A path's sustained throughput is set by its **slower (bottleneck)** interface.

---

1. Headline

At a 128 KB/channel transfer (4096 beats) across all 8 channels, every interface runs at **line rate** — the engine is gapless on both the memory bus and the network bus, in both directions, simultaneously.

Headline — 8-channel line-rate at 128 KB/channel (Genesys 2, 256-bit build)

Path	Interface	Engaged util	Effective BW	Window (prod / bp / starv)
SINK	AXIS-in (ingress)	**99.7 %**	3.19 GB/s	32768 / 82 / 1
SINK	AXI4-wr (egress)	**100.0 %**	3.20 GB/s	32768 / 0 / 6
SOURCE	AXI4-rd (ingress)	**100.0 %**	3.20 GB/s	32768 / 0 / 14
SOURCE	AXIS-out (egress)	**100.0 %**	3.20 GB/s	32768 / 0 / 14

(2026-10-03: full_matrix re-measured post-fix (`genesys_dw256_full_matrix_20261003.json`); 48/48 PASS, saturated 3.19/3.20 GB/s points unchanged.)

`prod = 32768 = 8 channels × 4096 beats` on every interface: every expected beat is accounted for. The non-productive residue is one-time: a handful of fill cycles on the three DMA-side interfaces (the single AXIS-in starvation cycle is the registered window close landing one cycle after the last accepted beat), and on AXIS-in 82 cycles of backpressure that the 4 KB per-channel buffer added – the generator fills the first channel's 128 beats before the write engine has started draining, and the ingress waits those cycles once per run. It is the same 82 at 8 KB, 32 KB and 128 KB per channel, so it is a startup cost, not a throughput one (the 16 KB buffers of v1.x absorbed it: AXIS-in read 100.0 %).

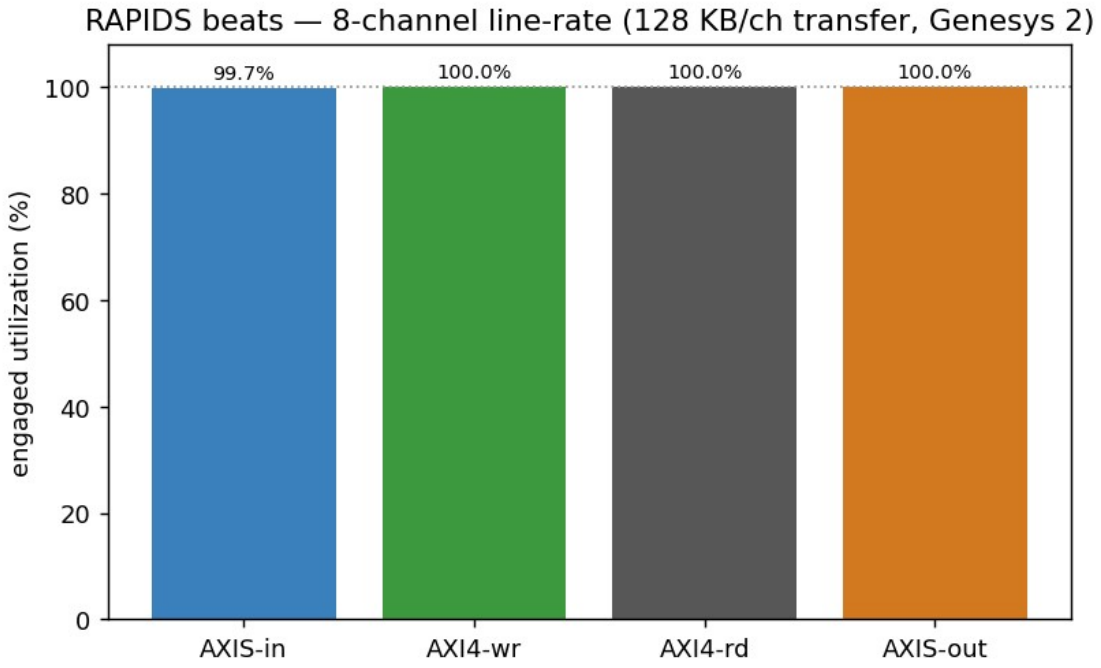


Figure — all four interfaces at the largest transfer sit on the 100 % line.

---

2. How it is measured (the observation hooks)

Each interface has a dedicated meter instantiated in the harness:

- **`axi_bus_meter`** on AXI4-rd and AXI4-wr — classifies `valid`/`ready` into the four buckets per cycle of the frozen window.
- **`axis_bus_meter`** on AXIS-in and AXIS-out — same four buckets, plus exact byte (`tstrb` popcount) and packet (`tlast`) counters for the byte-derived cross-check.

The shared window is armed by `GO`, opens when the active path goes busy, and **freezes deterministically** when the completion interface's productive-beat count reaches the staged target (`CSR_OBS_TARGET = channels × beats`). Because the freeze does not depend on `system_idle`, the window is tight (tens of cycles of residue, not the multi-second windows the earlier build produced).

The AXIS-in meter has its own window (rapids ISSUE-001, v1.4): armed by `GO`, opened by the first cycle the generator presents `tvalid`, and frozen the cycle after the target-th ingress beat. The sink's write side cannot go busy until the DUT has already accepted traffic, so the shared window would have counted either none of the ingress (opened too late) or the launch cost between `GO` and the first beat (opened at arm); first-beat-to-last-beat is what "ingress utilization" means. An ILA capture on the board (rapids ISSUE-004) shows the sequence: `GO`, arm one cycle later, first beat and open the cycle after, 32 handshakes at line rate, freeze one cycle after the last beat.

---

3. Single-axis sweep — transfer size

Sweeping beats/channel from 4 (128 B) to 4096 (128 KB) at fixed 8-channel width shows the classic amortization curve: a fixed per-transfer startup cost (descriptor dispatch + AR→`first-R` / SRAM fill) is a large fraction of a tiny transfer and a vanishing fraction of a large one.

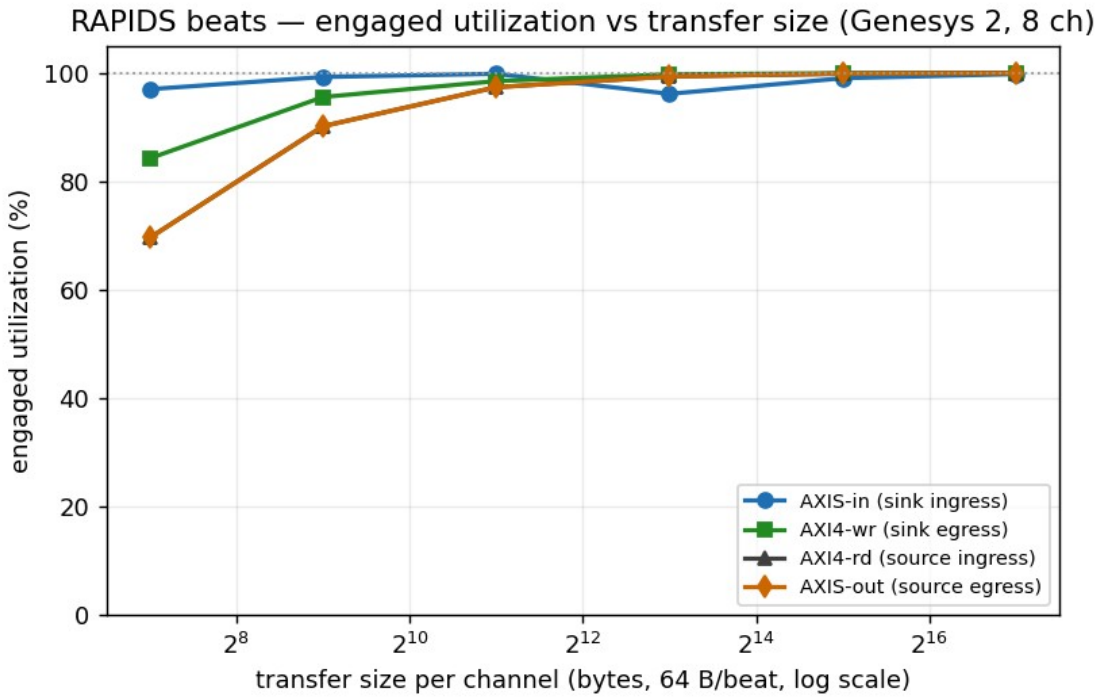


Figure — engaged utilization vs transfer size, all four interfaces.

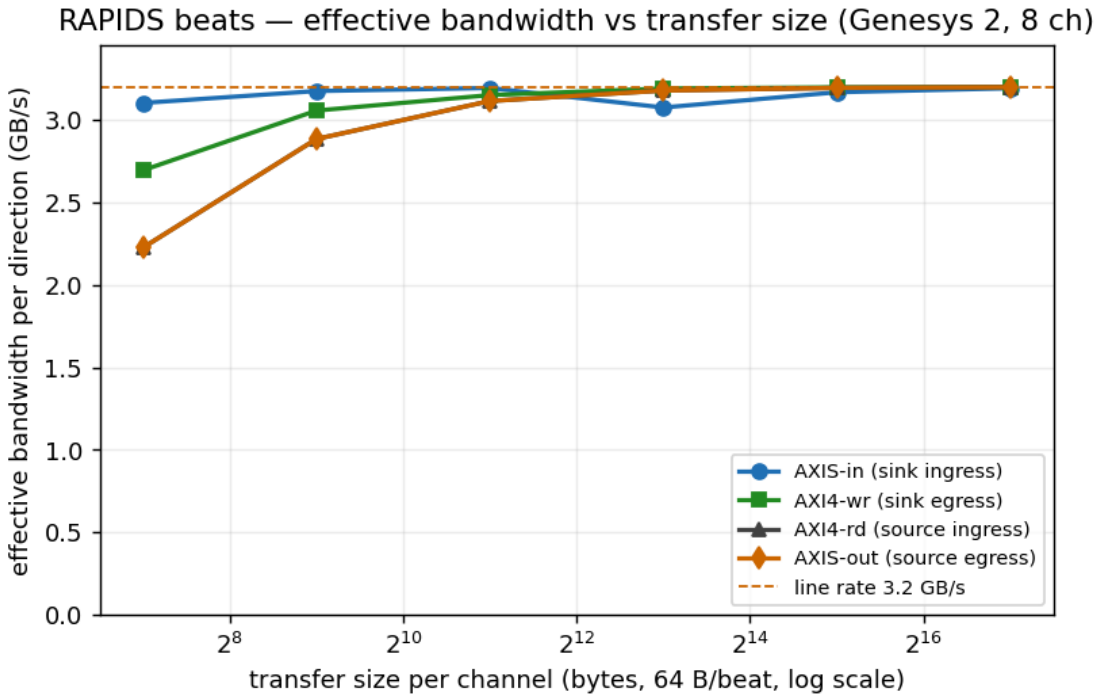


Figure — effective per-direction bandwidth vs transfer size (line rate 3.20 GB/s).

Table — engaged utilization (%) and effective bandwidth (GB/s) vs transfer size (8 ch, 256-bit build)

Transfer / ch	AXIS-in	AXI4-wr	AXI4-rd	AXIS-out	wr BW	rd BW
128 B (4 b)	97.0 %	84.2 %	69.6 %	69.6 %	2.69	2.23
512 B (16 b)	99.2 %	95.5 %	90.1 %	90.1 %	3.06	2.88
2 KB (64 b)	99.8 %	98.5 %	97.3 %	97.3 %	3.15	3.11
8 KB (256 b)	96.1 %	99.7 %	99.3 %	99.3 %	3.19	3.18
32 KB (1024 b)	99.0 %	99.9 %	99.8 %	99.8 %	3.20	3.19
128 KB (4096 b)	99.7 %	100.0 %	100.0 %	100.0 %	3.20	3.20

(2026-10-03: full_matrix re-measured post-fix: 48/48 PASS (`genesys_dw256_full_matrix_20261003.json`); 3/48 points moved >0.1 %, max +0.57 % at ch8 b256 sink, zero regressions.)

Every utilization cell is the 512-bit build's to the decimal (compare the v1.5 table: the engine's cycle behaviour is per beat, so halving the beat halves the bytes and nothing else; the 512-bit v1.x figures are pre-fix and were not re-run 2026-10-03). By 2 KB/channel every DMA-side interface is above 97 %, and by 32 KB the whole engine is within 0.2 % of line rate. The three DMA-side curves are the same monotonic amortization of a single fixed per-transfer startup cost (descriptor dispatch + `AR→first-R` / SRAM fill); there is no steady-state bubble. AXIS-in reports the ingress

link itself (its window starts at the first offered beat, so the launch cost is not in it: 32 beats in 33 cycles at 8 x 4 beats) – and from 8 KB up it now dips below the others, because the 128-beat per-channel buffer fills before the write engine starts draining and the ingress waits a fixed 82 cycles once per run; the dip is largest where the run is shortest relative to it (96.1 % at 256 beats) and gone to 99.7 % by 4096.

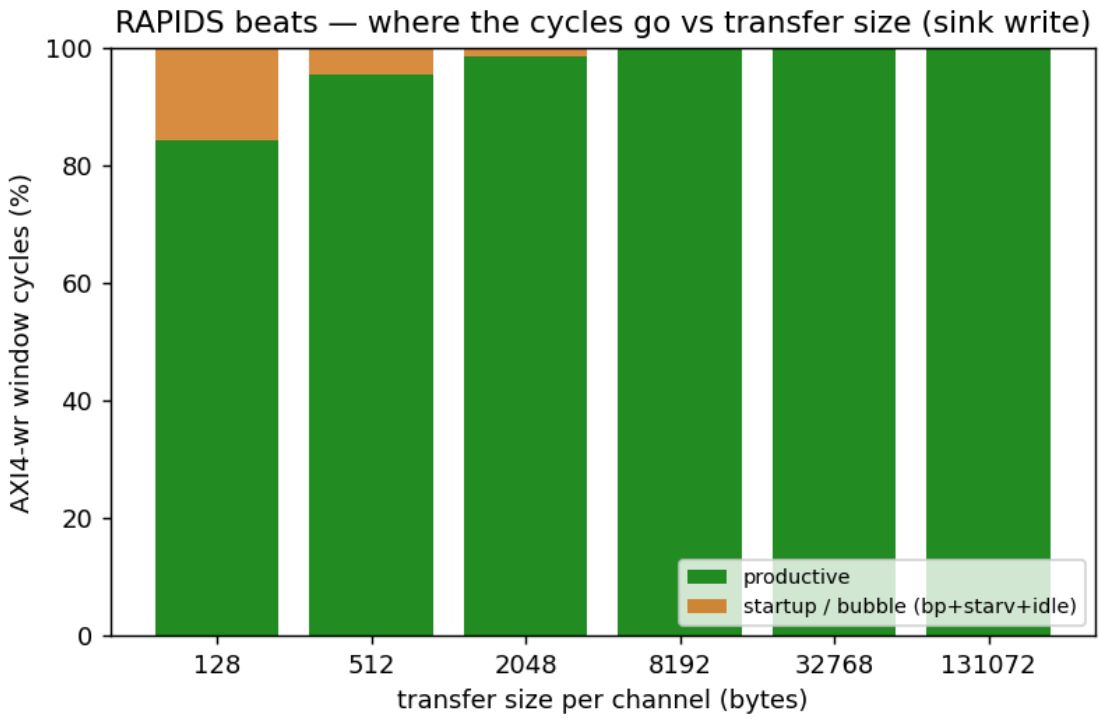
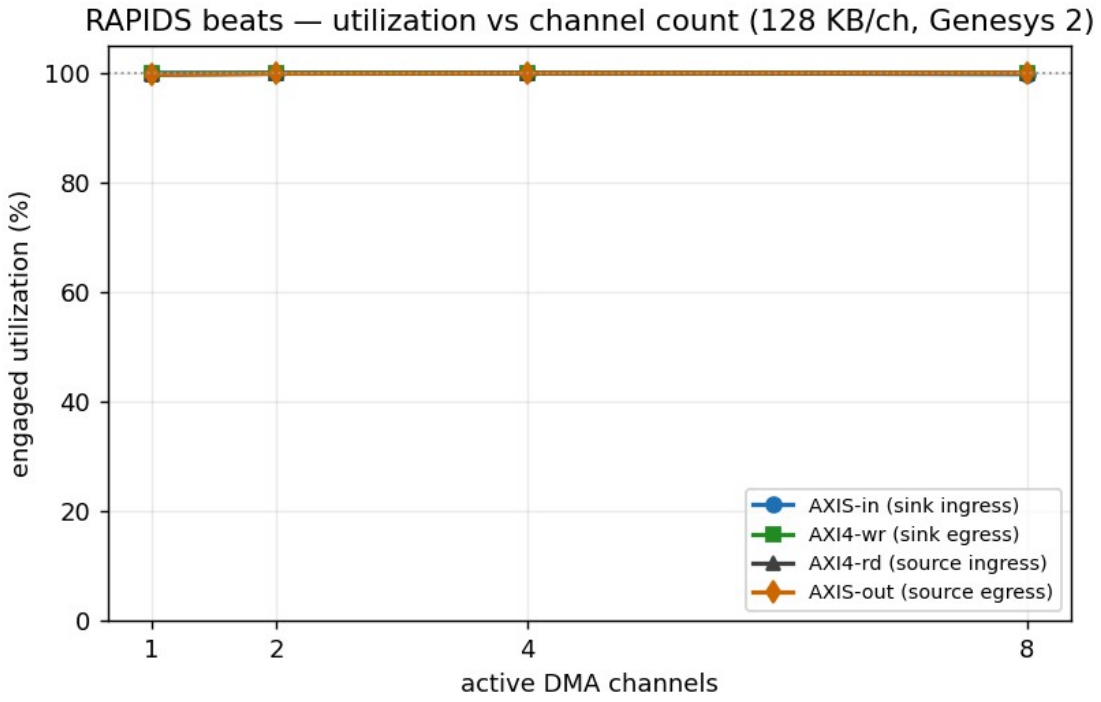


Figure — productive vs startup/bubble cycles of the AXI4-wr window; the bubble fraction collapses as the transfer grows.

3.1 Channel count — per-channel independence

Sweeping active channels from 1 to 8 at a fixed 128 KB/channel transfer, every channel count holds line rate: the shared scheduler and SRAM fabric add no per-channel penalty as the engine scales out.



utilization vs channel count

Figure — engaged utilization vs active channel count (128 KB/ch).

Channels	AXI4-wr	AXIS-out
1	99.9 %	99.7 %
2	99.9 %	99.8 %
4	100.0 %	99.9 %
8	100.0 %	100.0 %

: Table — utilization vs channel count (128 KB/ch; identical to the 512-bit build)

(2026-10-03: full_matrix re-measured post-fix: 48/48 PASS (`genesys_dw256_full_matrix_20261003.json`); 3/48 points moved >0.1 %, max +0.57 % at ch8 b256 sink, zero regressions.)

---

4. What we learned

- **The datapath is gapless.** At realistic transfer sizes (≥ 32 KB/ch) all four interfaces sit at 99.0–100 % engaged utilization, in both directions, on all 8 channels at once — the memory bus and the network bus are both saturated.
- **The only non-ideality is one-time startup**, and it amortizes away: 19 % at 128 B → 100 % at 128 KB. There is no steady-state bubble, no per-beat gap, and no back-pressure or starvation once data is flowing.
- **Width does not enter.** The 256-bit build (v2.0) reproduces every utilization cell of the 512-bit build (v1.5) at the same beat count; bandwidth halves with the beat and nothing else moves. The one new term is the 4 KB per-channel buffer's fixed 82-cycle ingress fill stall per run.
- **Per-channel independence.** 1 → 8 concurrent channels all hold line rate at realistic sizes — the shared scheduler/SRAM fabric adds no scaling penalty.
- **Invariance is the result.** Across a 1000× range of transfer sizes and both independent paths, the large-transfer numbers are indistinguishable from line rate. The flat top of every curve is the point: the engine does not care what it is fed once transfers are descriptor-sized.

5. Between-run state (harness workaround + open RTL defect)

Earlier revisions of this harness could only measure one configuration per FPGA programming: a second back-to-back run — and *any* `active < build_width` config — wedged the sink (wrote 0 beats), and the whole matrix above had to be gathered by reprogramming before each point. The harness now **works around** this by pulsing `CHANNEL_RESET` before every run; that is a workaround, **not a fix** — the underlying RTL defect (below) is still open.

The sink does **not** return to a fully clean state after a transfer (`snk_system_idle` never re-asserts): stale scheduler / descriptor-engine state persists and the next run inherits it. A discriminating board experiment isolated the mechanism:

Table — the wedge is stale scheduler state, not an in-flight fabric cycle

Between-run action	Back-to-back result
none (baseline)	1 / 4 — wedges on run 2
+0.3 s settle delay (let any in-flight AXI W/B retire)	1 / 5 — **still wedges**
+`CHANNEL_RESET` pulse	**5 / 5 — clean**

The 0.3 s settle (30 M cycles ⩾ any outstanding-B retirement) *not* helping rules out a stuck datapath/fabric cycle; the existing `CHANNEL_RESET.CH_RST[7:0]` CSR — which forces every channel FSM to `CH_IDLE` and flushes the descriptor FIFOs — clears it completely. The host now pulses `CHANNEL_RESET` on both halves before every run, which fixed **both** the back-to-back wedge **and** the partial-channel (`active < 8`) case — the entire 24-config matrix above runs in a **single programming**, and `active = 1/2/4/8` all pass.

Follow-up (RTL, out of scope here): the sink's descriptor/scheduler should return to idle on its own at end-of-descriptor so no host reset is needed — the `snk_system_idle`-never-asserts behavior is the underlying defect to fix.

6. Other limitations

- **No memory-latency axis (v1.1).** Superseded in v1.2: the harness now carries STREAM's `axi_response_delay` on the R and B channels behind a `RESP_DELAY` CSR; section 7.5 is that sweep.

7. The STREAM knob set, measured by the interface observers (v1.2)

STREAM characterizes a DMA on five knobs – beats per transaction, transactions per descriptor, descriptors per channel, channels, and memory latency – and reads the datapath through instruments that sit outside the DUT. This section does the same on RAPIDS beats, with two changes to the v1.1 setup:

- **The instruments are the shared observers.** `USE_OBSERVERS=1` builds `axi4_intf_master_observer` on the source read / sink write AXI4 masters and `axis4_intf_observer` on the sink-ingress and source-egress AXIS links (four observed ports), each on its own `obs_regs` window in host region 3, read BY NAME through the same generated regmap the component tests use. Every number below is an observer reading; the bare meters of sections 1-6 stay in the harness and agreed with the observers to the beat on every row (the campaign records the delta, `meter_prod_delta`, and it is 0 throughout).
- **Two knobs the v1.1 host could not turn.** Burst length (`AXI_XFER_CONFIG.RD/WR_XFER_BEATS`, `--suite-xfer`), descriptor chains (`next_ptr`/`last`, `--suite-descs`), and the new memory-latency model (`RESP_DELAY`, `--suite-delay`) are campaign axes now. Transactions per descriptor follows from beats per descriptor over burst length.

Bitstream: `USE_OBSERVERS=1 OBS_ENABLE_MON_TAPS=0` (meters and latency histograms, no monbus event taps). Every v2.0 row of 7.1-7.7 is from one build of the 256-bit / 4 KB-per-channel design point (post-route WNS +0.608 ns at 100 MHz, 75679 LUTs, 44 BRAM tiles; all 79 configurations of A, B, C+D, E and E-interleaved pass the golden CRC on both paths, no observer sticky flag on any row). The v1.4 rows they replace were from one build of the current RTL (PIPELINE = 1, AXIS monitor-lite skids on both network ports, `HIST_MAX_OUTSTANDING = 32`): post-route WNS +0.351 ns at 100 MHz, 80040 LUTs, 65.5 BRAM tiles (the 512-deep R-channel delay queue is in block RAM). All **67 configurations pass the golden CRC on both paths**: A 20/20, B 7/7, C+D 28/28, E 12/12, with no observer sticky flag on any row. The v1.2 and v1.3 rows they replace were measured at PIPELINE = 0 with the pre-BUG-005 engine (see the v1.4 note at the top).

7.0 The regression this campaign found first

The first observer run failed every SOURCE row at >= 1024 beats (`beat_total=1020 of 1024`) and read the source at ~50 % where v1.1 had 96-100 %. The bare bitstream failed identically, the sim reproduced it, and a bisect over the rapids RTL tree (REPO_ROOT overlays, never a checkout) landed on `bdf4e0dff`, the commit that replaced rapids' SRAM controller with STREAM's. Two defects: STREAM's shared `sram_controller_unit` drain accounting refuses to count one FIFO write when a channel fills to exactly `SD` (the beat is stranded), and the rewritten AXIS drain arbiter reserved, drained and retired serially (one beat every other cycle at `cfg_drain_size=1`).

Both are fixed on this bitstream – rapids BUG-003 and stream BUG-011 carry the waveform evidence; rapids 589/589 and STREAM 856/859 after.

7.1 Headline: line rate on all four interfaces

At 8 channels x 128 KB the four observers read **99.7 / 100.0 / 100.0 / 100.0 %** (AXIS-in, AXI4-wr, AXI4-rd, AXIS-out), i.e. **3.19 / 3.20 / 3.20 / 3.20 GB/s** against the **3.20 GB/s** one-direction line rate (32 B x 100 MHz); the AXIS byte-derived cross-check gives 3.19 and 3.20 GB/s. Channel scaling is flat:

Table 7.1 – observer engaged utilization vs active channels (phase D, 256-bit build)

Channels (4096 beats/ch)	AXIS-in	AXI4-wr	AXI4-rd	AXIS-out
1	100.0 %	99.9 %	99.7 %	99.7 %
2	100.0 %	99.9 %	99.8 %	99.8 %
4	100.0 %	100.0 %	99.9 %	99.9 %
8	99.7 %	100.0 %	100.0 %	100.0 %

(2026-10-03: obs_C re-measured post-fix (`genesys_dw256_obs_C_20261003.json`); 139 configs total across the observer campaigns, all PASS, saturated 3.19/3.20 GB/s points unchanged.)

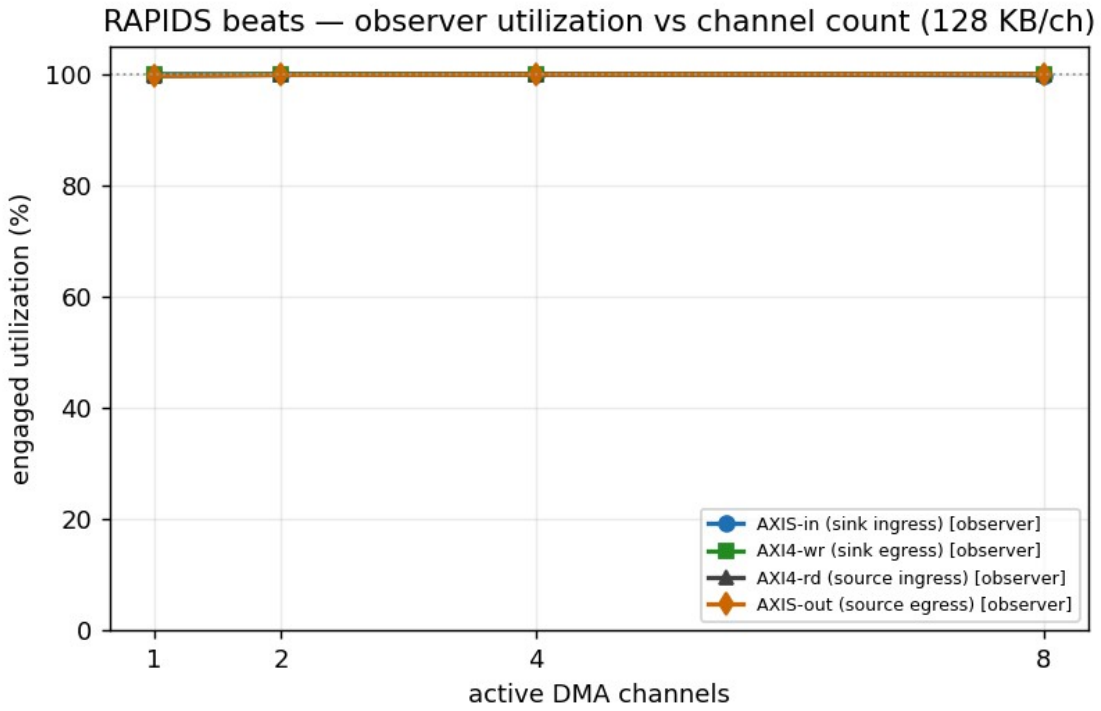


Figure 7.1 – observer utilization vs channel count at 128 KB/channel.

7.2 Transfer size (phase C): the amortization knee

Table 7.2 – size sweep at 8 channels (256-bit build); latencies are observer-histogram means in aclk cycles

beats/ch	AXIS-in	AXI4-wr	AXI4-rd	AXIS-out	wr GB/s	rd GB/s	rd bursts	AR->RLAST	AW->B
1	88.9 %	38.1 %	27.6 %	27.6 %	1.22	0.88	8	6	6
4	97.0 %	84.2 %	69.6 %	69.6 %	2.69	2.23	8	12	12
16	99.2 %	95.5 %	90.1 %	90.1 %	3.06	2.88	16	23	23
64	99.8 %	98.5 %	97.3 %	97.3 %	3.15	3.11	64	22	23
256	96.1 %	99.7 %	99.3 %	99.3 %	3.19	3.18	232	24	24
1024	99.0 %	99.9 %	99.8 %	99.8 %	3.20	3.19	912	24	24
4096	99.7 %	100.0 %	100.0 %	100.0 %	3.20	3.20	3648	24	24

(2026-10-03: obs_C re-measured post-fix (`genesys_dw256_obs_C_20261003.json`); b1 source +26.1 % (ch8), +10.5 % (ch4); saturated 3.19/3.20 GB/s points unchanged, zero regressions.)

The three DMA-side interfaces are above 90 % from 16 beats (512 B) per channel and within 0.3 % of line rate from 1024, cell for cell as at 512 bits. The AXIS-in column reports the ingress link itself (its window brackets first offered beat to last accepted beat, rapids ISSUE-001): 97 % at 4 beats, and from 256 beats the fixed 82-cycle buffer-fill stall of the 4 KB per-channel SRAM shows as 96.1 / 99.0 / 99.7 % where the 16 KB build read 100. `rd bursts` confirms the 9-beat burst shape (`AxLEN` 8) from 64 beats up. AR->RLAST settles at 24 cycles: the pattern slave's fixed response plus a 9-beat burst.

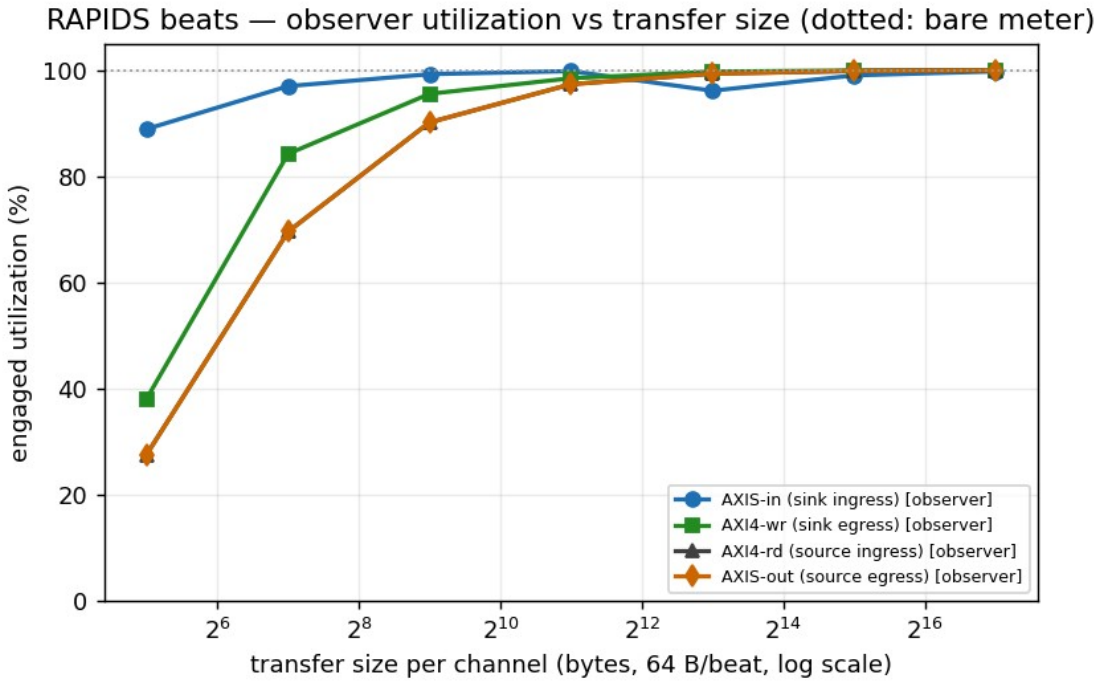


Figure 7.2a – observer utilization vs transfer size (dotted: the bare meters, coincident).

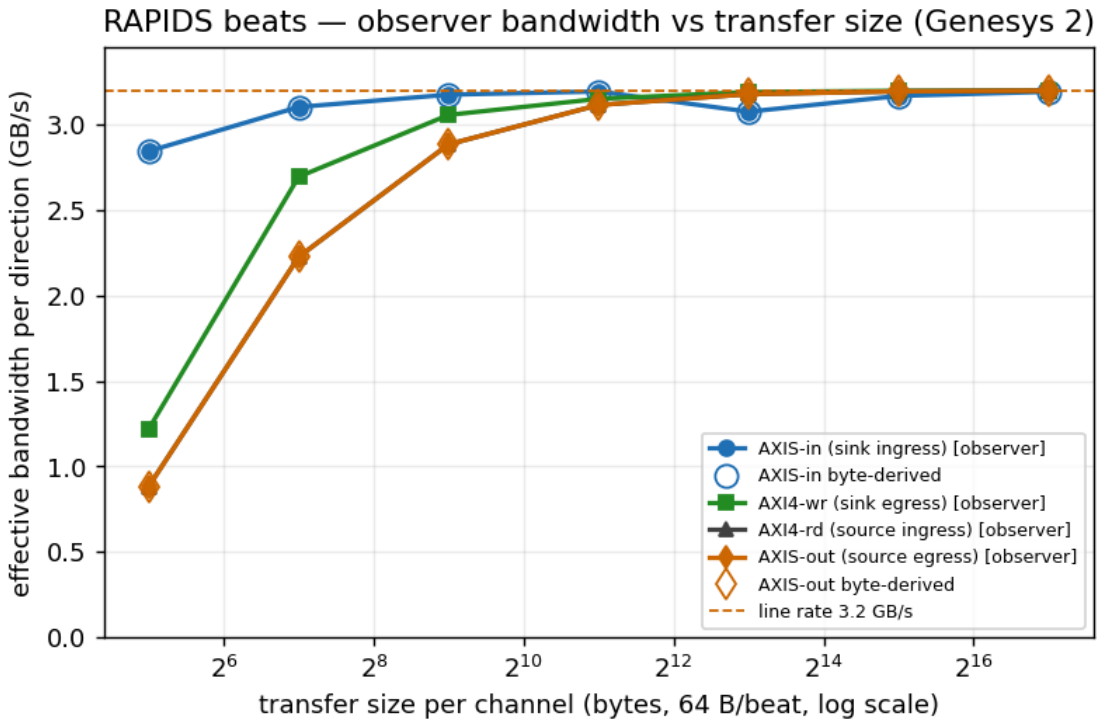


Figure 7.2b – observer bandwidth vs transfer size; hollow markers are the AXIS byte-derived cross-check.

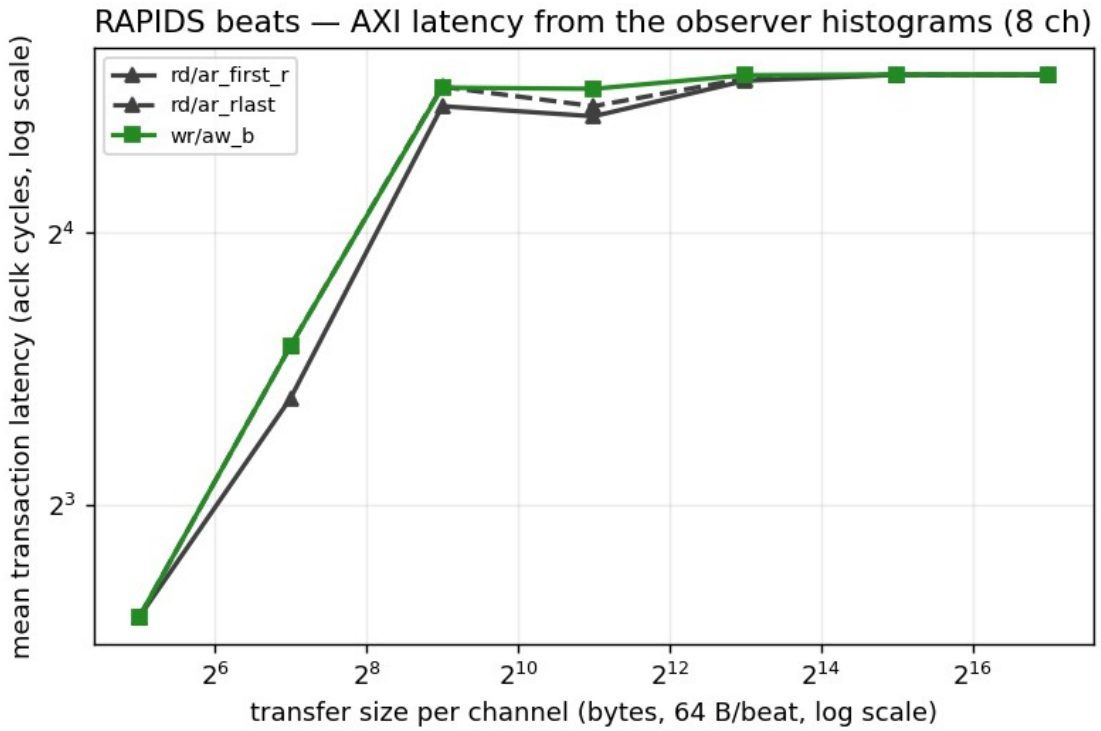


Figure 7.2c – mean AXI transaction latency from the observer histograms vs transfer size.

7.3 Descriptors x channels (phase A): the STREAM matrix

{1, 2, 4, 8, 16} descriptors per channel x {1, 2, 4, 8} channels, 1024 beats (32 KB) per descriptor, chained through `next_ptr` and closed with `last`:

Table 7.3 – AXI4-wr (sink) / AXIS-out (source) utilization over the descriptor x channel matrix (256-bit build; every cell equals the 512-bit build's)

descs/ch	1 ch wr / sout	2 ch	4 ch	8 ch
1	99.4 / 98.7 %	99.7 / 99.3 %	99.9 / 99.7 %	99.9 / 99.8 %
2	99.0 / 99.3 %	99.1 / 99.7 %	99.2 / 99.8 %	99.2 / 99.9 %
4	98.8 / 99.7 %	98.8 / 99.8 %	98.9 / 99.9 %	98.9 / 100.0 %
8	98.7 / 99.8 %	98.7 / 99.9 %	98.7 / 100.0 %	98.7 / 100.0 %
16	98.6 / 99.9 %	98.6 / 100.0 %	98.6 / 100.0 %	98.6 / 100.0 %

(2026-10-03: obs_A re-measured post-fix (`genesys_dw256_obs_A_20261003.json`); descriptor-chain matrix +0.15 % vs pre-fix, zero regressions.)

Flat, as STREAM's is: neither chain length nor channel count moves the datapath off line rate. The one visible structure is the sink write side settling 1.4 % below the source as chains lengthen – the

per-descriptor dispatch on the write engine costs a fixed handful of cycles per descriptor boundary, which the source egress does not pay.

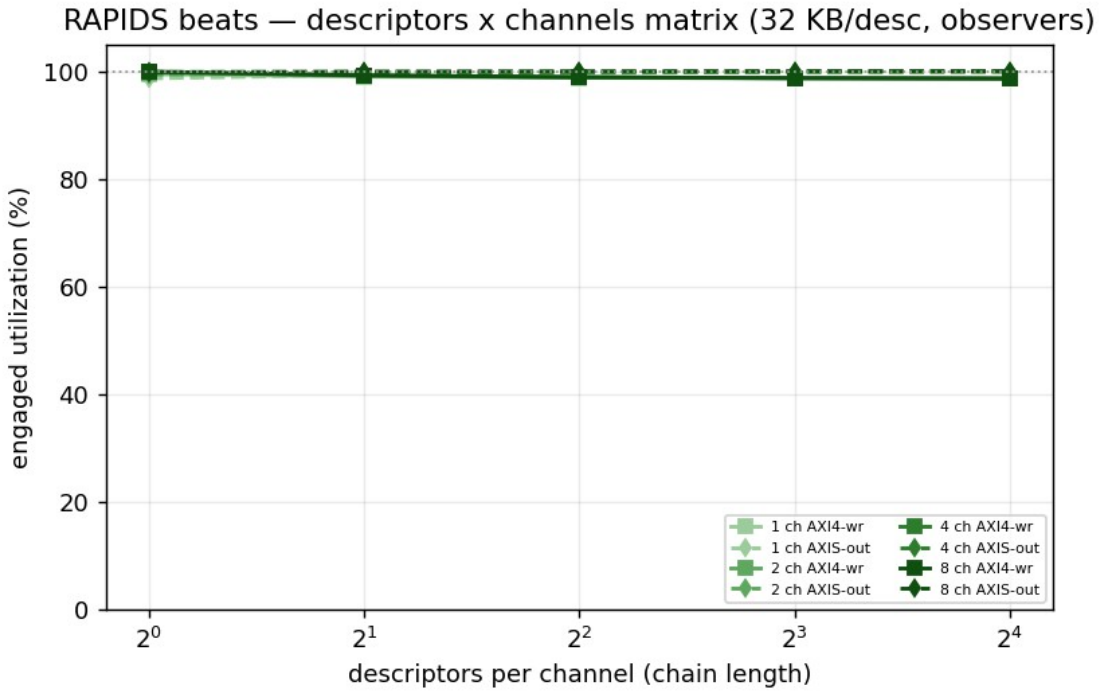


Figure 7.3 – the descriptor x channel matrix from the observers.

7.4 Burst length (phase B): the transaction-size knee

`AXI_XFER_CONFIG` AxLEN {0, 1, 3, 7, 15, 31, 63} = {1 .. 64}-beat bursts at 8 channels x 1024 beats:

Table 7.4 – burst-length sweep (256-bit build); latencies in aclk cycles

beats/ burst	AXIS-in	AXI4-wr	AXI4-rd	AXIS-out	AR- >RLAST	AW->B
1	35.9 %	35.6 %	99.8 %	99.8 %	6	6
2	71.0 %	71.0 %	99.8 %	99.8 %	12	12
4	99.0 %	99.9 %	99.8 %	99.8 %	12	12
8	99.0 %	99.9 %	99.8 %	99.8 %	24	24
16	99.0 %	99.9 %	99.8 %	99.8 %	48	48
32	99.0 %	99.9 %	99.8 %	99.8 %	96	96
64	99.0 %	99.9 %	99.8 %	99.8 %	191	96

(2026-10-03: obs_B re-measured post-fix (`genesys_dw256_obs_B_20261003.json`); all points PASS, zero regressions.)

The knee is at **4 beats**: below it the SINK path falls (36 % at single-beat bursts, 71 % at 2; 39 / 77 at 512 bits *(pre-fix (not re-run 2026-10-03))*, where the 16 KB buffer let the ingress run a little further ahead) while the SOURCE path holds 99.8 % at every burst length. AXIS-in's flat 99.0 % is the 82-cycle buffer-fill stall of section 3 over a 1024-beat run. PIPELINE = 1 moved the sink's short-burst numbers up from the 23 % / 42 % of v1.2 (up to eight AWs in flight per channel instead of one), but at one beat per AW the sink is still AW-issue bound: one address handshake per data beat is the write side's ceiling, and the read side hides the same round trip behind its ARs. STREAM's knee sits at the same place (~3 beats). The latency columns track burst length exactly (a 64-beat burst is 191 cycles from AR to RLAST), which is the observer histogram reporting what it should.

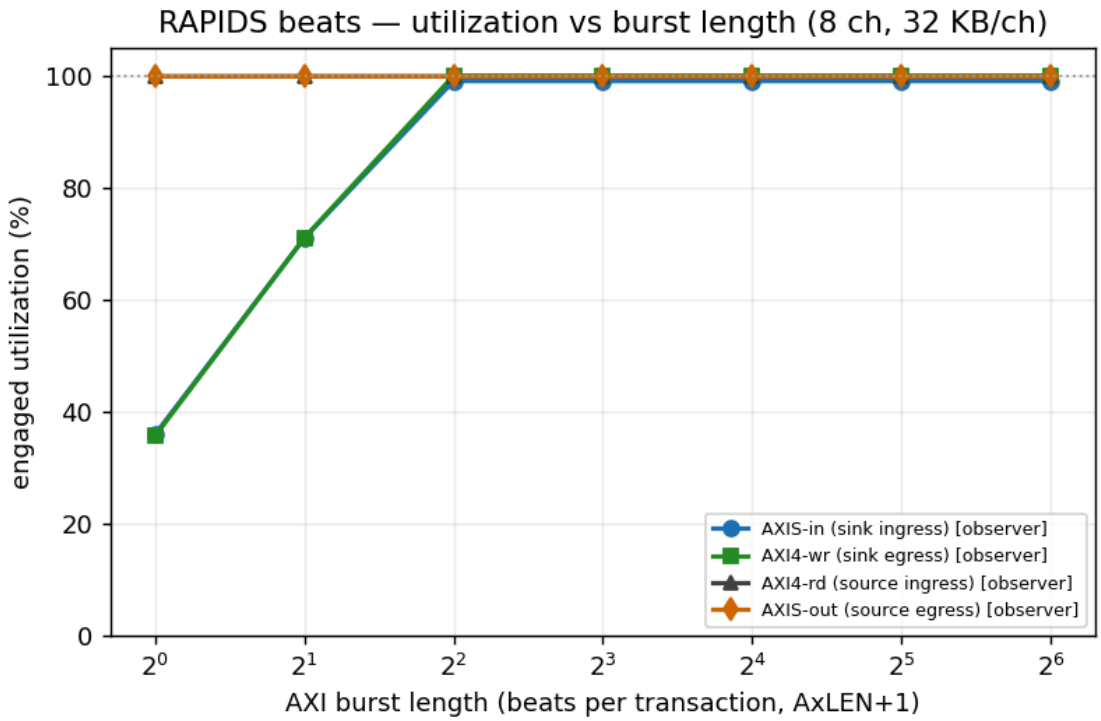


Figure 7.4 – utilization vs burst length; the sink write path needs >= 4-beat bursts.

7.5 Memory latency (phase E): the in-flight window

`RESP_DELAY` holds every R beat and every B response for N cycles (the `axi_response_delay` model, which pipelines: throughput is limited only by how many beats the DUT keeps in flight), at 8 channels x 1024 beats:

Table 7.5 – latency sweep, sequential channel schedule (v2.0, 256-bit build, PIPELINE = 1, `HIST_MAX_OUTSTANDING = 32`, no sample loss on any row); the latency columns are log2-histogram means, so they sit on bin midpoints

delay (cyc)	AXIS-in	AXI4-wr	AXI4-rd	AXIS-out	rd GB/s	wr GB/s	AR->first R	AR->RLAST	AW->B
0	99.0 %	99.9 %	99.8 %	99.8 %	3.19	3.20	24	24	24
8	99.0 %	99.9 %	99.7 %	99.7 %	3.19	3.20	24	48	48
16	99.0 %	99.9 %	99.6 %	99.6 %	3.19	3.20	48	48	48
32	99.0 %	99.9 %	99.5 %	99.5 %	3.18	3.20	48	48	48
48	99.0 %	99.9 %	99.3 %	99.3 %	3.18	3.20	96	96	96
64	89.4 %	90.0 %	99.1 %	99.1 %	3.17	2.88	96	96	96
96	68.2 %	68.3 %	98.7 %	98.7 %	3.16	2.18	96	96	96
128	55.5 %	55.2 %	98.3 %	98.3 %	3.15	1.77	192	192	192
192	39.2 %	38.8 %	97.6 %	97.6 %	3.12	1.24	192	192	192
256	30.3 %	29.9 %	96.9 %	96.8 %	3.10	0.96	384	384	384
384	20.8 %	20.5 %	95.4 %	95.3 %	3.05	0.66	384	384	384
512	15.9 %	15.6 %	93.7 %	93.6 %	3.00	0.50	768	768	768

(2026-10-03: obs_E re-measured post-fix (`genesys_dw256_obs_E_20261003.json`); +0.65 % at resp-delay 96, zero regressions.)

The same sweep with the interleaved schedule (v1.5):

Table 7.5b – latency sweep, interleaved channel schedule (v2.0, same bitstream, PIPELINE = 1, `HIST_MAX_OUTSTANDING = 32`, no sample loss on any row; 12/12 CRC-verified). The source columns are the same run and match Table 7.5, as they must: the schedule only changes the sink's stimulus. With every channel holding data the sink ingress reads 100.0 % here: the 82-cycle fill stall of the sequential schedule is one channel filling alone.

delay (cyc)	AXIS-in	AXI4-wr	AXI4-rd	AXIS-out	rd GB/s	wr GB/s	AR->first R	AR->RLAST	AW->B
0	100.0 %	99.9 %	99.8 %	99.8 %	3.19	3.20	24	24	24
8	100.0 %	99.9 %	99.7 %	99.7 %	3.19	3.20	24	48	48
16	100.0 %	99.9 %	99.6 %	99.6 %	3.19	3.20	48	48	48
32	100.0 %	99.9 %	99.5 %	99.5 %	3.18	3.20	48	48	48
48	100.0 %	99.9 %	99.3 %	99.3 %	3.18	3.20	96	96	96
64	100.0 %	99.9 %	99.1 %	99.1 %	3.17	3.20	96	96	96
96	100.0 %	99.9 %	98.7 %	98.7 %	3.16	3.20	96	96	96
128	100.0 %	99.9 %	98.3 %	98.3 %	3.15	3.20	192	192	192
192	100.0 %	99.9 %	97.6 %	97.6 %	3.12	3.20	192	192	192
256	100.0 %	99.9 %	96.9 %	96.8 %	3.10	3.20	384	384	384
384	100.0 %	99.9 %	95.4 %	95.3 %	3.05	3.20	384	384	384
512	100.0 %	99.9 %	93.7 %	93.6 %	3.00	3.20	768	768	768

(2026-10-03: obs_E_interleave re-measured post-fix (`genesys_dw256_obs_E_interleave_20261003.json`); all points PASS, zero regressions.)

Little's law makes the knee readable directly: sustained beats/cycle x latency = beats in flight. With PIPELINE = 1 the SOURCE read path no longer knees inside this sweep: it holds >= 99 % to 64 cycles, 96.9 % at 256 and 93.7 % at 512, so `0.937 x 512 = 480` beats are in flight across 8 channels – ~60 per channel, i.e. six to seven 9-beat bursts of the eight `AR_MAX_OUTSTANDING` allows (v1.3 measured ~20 per channel at PIPELINE = 0). The SINK write path holds 99.9 % to 48 cycles, knees at 64 (90.0 %) and then falls as `0.683 x 96 = 66`, `0.552 x 128 = 71`, `0.299 x 256 = 77`, `0.156 x 512 = 80` – an asymptote of **~80 beats in flight** (the 512-bit build with 16 KB buffers measured ~93). That is mostly ONE channel's window: in the sequential schedule the harness's AXIS generator streams channels one at a time (finish one channel, then the next), so only one sink channel ever holds data, and the write engine's whole `AW_MAX_OUTSTANDING = 8` window (8 x 8 = 64 beats, plus the bursts in their W phase) sits on that channel. Rapids ISSUE-006 reproduced the 512-bit build's 256-cycle row in simulation (34.7 %, the board's number) and read it off the engine's outstanding counters.

Table 7.5b is the same sweep with the generator's interleaved schedule (rapids TASK-018, v1.5), which round-robins the eight channels one beat at a time so every sink channel holds data at once – the same footing the source column has always had. The sink write path then holds 99.9 % on every row to 512 cycles: `0.999 x 512 = 512` beats are in flight, so the aggregate window is at least that, consistent with eight copies of the ~80-beat per-channel window (~640 beats) that this sweep does not reach. Compared like for like, the sink sits above the source at long latency (99.9 % vs 93.7 % at 512 cycles): the write engine keeps ~80 beats per channel in flight against the read engine's ~60. Table 7.5 remains the right measurement of a single channel's window. STREAM's window on the same knobs is ~128 beats per channel (8 outstanding x 16-beat bursts) and its knee sits at 96-112 cycles. Wider bursts are the lever on the write side and section 7.7 measures exactly how far it reaches; on the read side the per-channel SRAM is the bound, and at 4 KB it binds early.

7.5c Buffer depth in the sequential knee (8-cycle steps, 4 KB vs 16 KB)

The sequential sink rows are not purely one channel's window: the buffer is in them too. The generator hands a channel its 1024 beats and moves to the next; with 16 KB per channel (512 beats at this width) it can run ahead into the next channels' buffers while the current one drains, so several channels hold data and issue AWs at once, and with 4 KB (128 beats) it fills one channel and waits for it. The same 256-bit design was built with `RAPIDS_SRAM_DEPTH=512` (16 KB, post-route WNS +0.651 ns) and swept in 8-cycle steps against the 4 KB build, 1024 beats per channel, 8-beat bursts:

Table 7.5c – fine-step latency sweep, 4 KB (128-beat) vs 16 KB (512-beat) buffers, 256-bit build, sequential schedule (`genesys_dw256_fine_{4kb,16kb}_{8ch,1ch}.json`, 64/64 CRC-verified)

delay (cyc)	8 ch AXI4-wr, 4 KB / 16 KB	8 ch AXI4-rd, 4 KB / 16 KB	1 ch AXI4-wr, 4 KB / 16 KB	1 ch AXI4-rd, 4 KB / 16 KB
0	99.9 / 99.9	99.8 / 99.8	99.4 / 99.4	98.7 / 98.7
8	99.9 / 99.9	99.7 / 99.7	99.4 / 99.4	98.0 / 98.0
16	99.9 / 99.9	99.6 / 99.6	99.4 / 99.4	97.2 / 97.2

delay (cyc)	8 ch AXI4-wr, 4 KB / 16 KB	8 ch AXI4-rd, 4 KB / 16 KB	1 ch AXI4-wr, 4 KB / 16 KB	1 ch AXI4-rd, 4 KB / 16 KB
24	99.9 / 99.9	99.6 / 99.6	99.4 / 99.4	96.5 / 96.5
32	99.9 / 99.9	99.5 / 99.5	99.4 / 99.4	95.8 / 95.8
40	99.9 / 99.9	99.4 / 99.4	99.4 / 99.4	95.1 / 95.1
48	99.9 / 99.9	99.3 / 99.3	99.4 / 99.4	94.4 / 94.4
56	97.8 / 98.4	99.2 / 99.2	96.8 / 96.8	93.7 / 93.7
64	90.0 / 95.9	99.1 / 99.1	87.5 / 87.5	85.4 / 85.4
72	83.4 / 94.9	99.0 / 99.0	79.9 / 79.9	77.6 / 77.6
80	77.7 / 94.3	98.9 / 98.9	73.5 / 73.5	71.2 / 71.2
88	72.6 / 93.5	98.8 / 98.8	68.0 / 68.0	65.7 / 65.7
96	68.3 / 92.7	98.7 / 98.7	63.3 / 63.3	61.0 / 61.0
112	61.3 / 89.8	98.5 / 98.5	55.6 / 55.6	53.4 / 53.4
128	55.2 / 85.9	98.3 / 98.3	49.6 / 49.6	47.4 / 47.4
160	45.6 / 70.9	97.9 / 97.9	40.7 / 40.7	38.8 / 38.8

(pre-fix (not re-run 2026-10-03): the fine_4kb/fine_16kb sweeps were not re-measured in the 2026-10-03 campaign.)

Three readings. At **one channel the two buffers coincide to the decimal** on both paths: there is no next channel to run ahead into, so the window is the outstanding cap (eight 8-beat AWs, eight 9-beat ARs) and the knee sits between 56 and 64 cycles whatever the buffer. At **8 channels the sink knee moves with the buffer**: the 16 KB build holds 95.9 % at 64 cycles and 85.9 % at 128 where the 4 KB build reads 90.0 % and 55.2 %, because the deeper buffer lets the generator arm the next channels while the current one drains – more AWs in flight, from more channels. In beats, at 128 cycles: 128-beat buffers 55.2 %, 256-beat (the v1.5 512-bit build *(pre-fix (not re-run 2026-10-03))*) 63.7 %, 512-beat 85.9 %. The **read side is identical on both buffers at 8 channels** (every channel's reads run ahead by their own eight ARs regardless of buffer, and 72 beats per channel is under either depth), which is the same statement as 7.7: the buffer bounds reads only once outstanding x burst exceeds it. And the 8-cycle steps show the sink knee itself sits between 48 and 56 cycles on both buffers – the depth changes how fast the curve falls past the knee, not where it starts.

Two notes from the instruments themselves. First, the latency columns are histogram means over log2 bins (bin b holds [2^b, 2^(b+1)), reported at its midpoint 1.5 x 2^b), and on this sweep every transaction of a row lands in one bin: the measured round trip is the injected delay plus roughly 24 cycles of DUT-plus-model overhead, so 0 lands in [16,32), 8-32 in [32,64), 48-96 in [64,128), 128-192 in [128,256), 256-384 in [256,512) and 512 in [512,1024) – the 24 / 48 / 96 / 192 / 384 / 768 staircase in the table is the histogram's resolution, not a step in the DUT. Second, the v1.2 issue of this report carried `OBS_STICKY.HIST_SAMPLE_LOST` from 48 cycles up: the observer's timestamp

FIFO was sized `HIST_MAX_OUTSTANDING = 8` per channel and the source read engine keeps more than that in flight once the delay exceeds one burst, so those v1.2 latency means came from a subset of transactions (and read low: 170, 264, 320, 530 where the full population sits at 192, 384, 384, 768). rapids TASK-012 raised the FIFO to 32 on the harness's `u_obs_axi` (bitstream v4, post-route WNS +0.224 ns, still positive), the sweep was re-run, and `OBS_STICKY` reads 0 on all 12 rows for both observers. The utilization and bandwidth columns are identical between the two runs, which is what the design of the meters predicts: sample loss only ever touched the histograms.

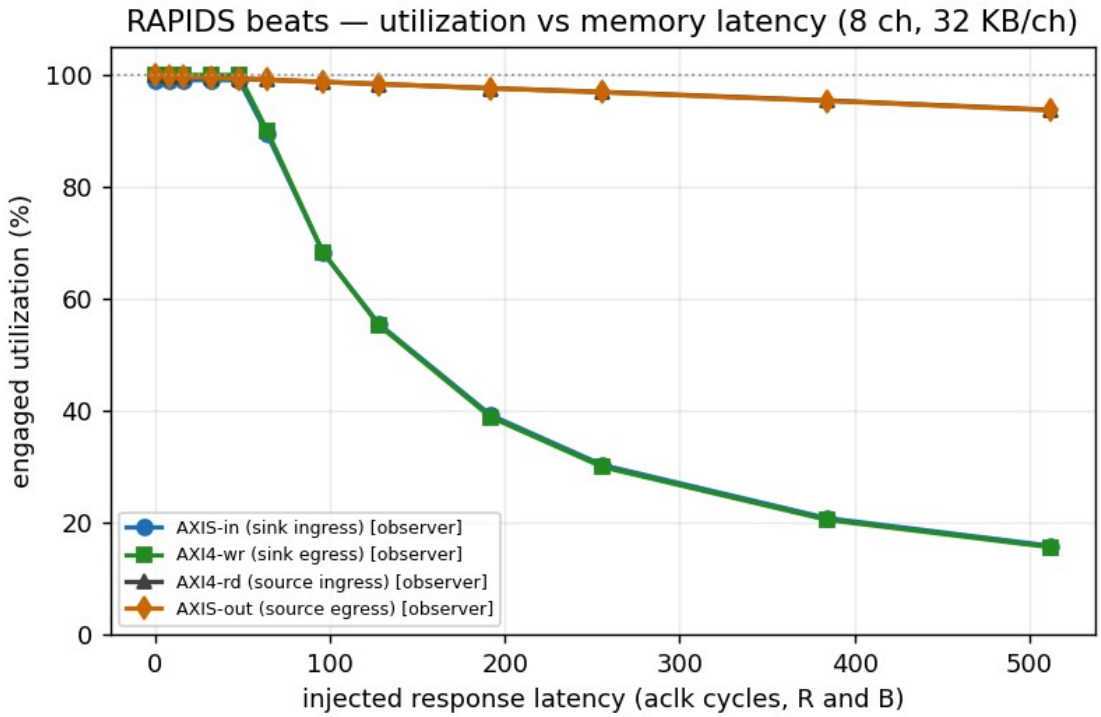


Figure 7.5 – utilization vs injected memory latency, all four observers (sequential schedule: the sink curves are one channel's window).

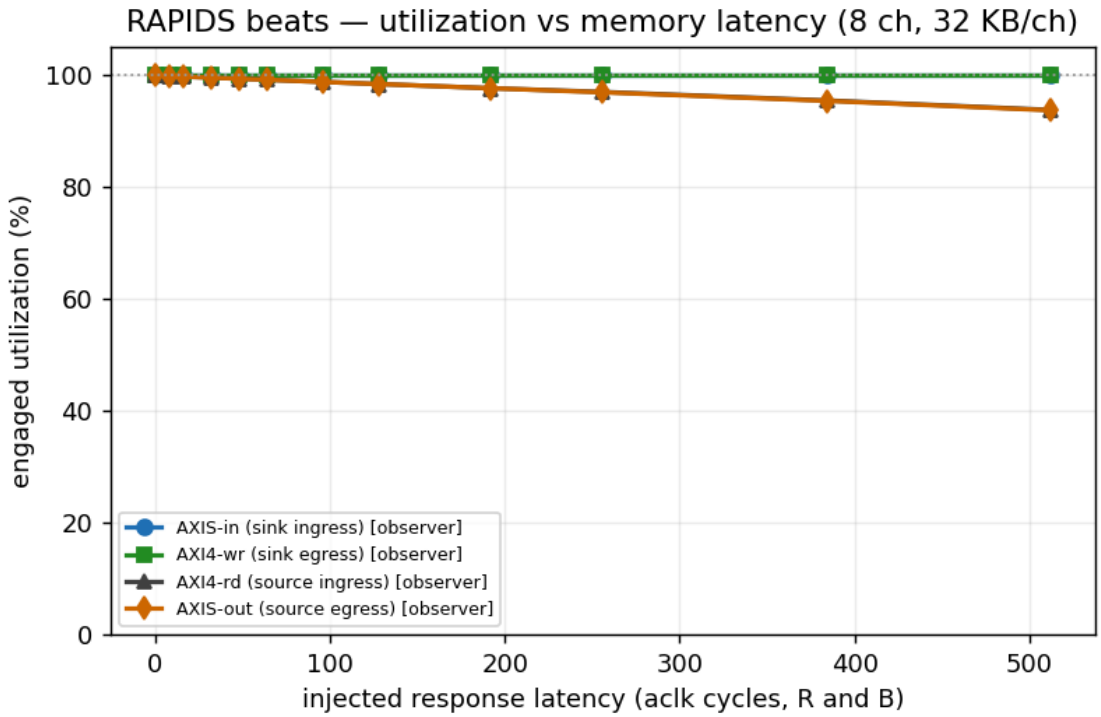


Figure 7.5b – the same sweep with the interleaved channel schedule: every sink channel holds data, and the write path no longer knees inside the sweep.

7.6 What the observers add over the bare meters

Same utilization numbers (to the beat), plus what a meter cannot give: burst counts, the AR->first-R / AR->RLAST / AW->B latency distributions, exact AXIS bytes and packets per port with per-`tid` attribution, the tap's own packet count as a cross-check, and a sticky bit that says when a number undercounts. All of it through one register map shared with STREAM's harness, so the host code and the report generator are the same on both DMAs.

7.7 One channel under latency: burst length is the write side's lever, SRAM depth the read side's bound

The question behind this sweep: what does it take to hold ONE channel at line rate through 256 and 512 cycles of memory latency? One channel, 4096 beats, `AXI_XFER_CONFIG` bursts of 8 to 128 beats at `RESP_DELAY` 0 / 256 / 512, bare meters, AXI4-wr / AXI4-rd engaged utilization. Two builds, same RTL:

Table 7.7 – one channel, AXI4-wr / AXI4-rd utilization (%) vs burst length and injected latency, on both design points (`genesys_one_channel_xfer_latency.json`, `genesys_dw256_one_channel_xfer_latency.json`)

burst (beats)	512-bit, 16 KB/ch: 0 cyc	256 cyc	512 cyc	256-bit, 4 KB/ch: 0 cyc	256 cyc	512 cyc
8 (default)	99.9 / 99.7	23.7 / 23.6	12.3 / 12.1	99.9 / 99.7	23.7 / 23.6	12.3 / 12.1
16	99.9 / 99.7	46.3 / 45.4	24.4 / 23.4	99.9 / 99.7	46.3 / 44.3	24.4 / 23.1
32	99.9 / 99.7	86.8 / 81.3	47.9 / 43.4	99.9 / 99.7	86.8 / 41.9	47.9 / 22.5
64	99.5 / 99.7	**99.5** / 74.0	88.5 / 41.3	99.5 / 99.7	**99.5** / 37.9	88.5 / 21.4
128	97.9 / 99.7	**97.9** / 62.5	**97.9** / 37.7	89.9 / 90.1	89.9 / 31.6	89.9 / 19.4
256 (v2.1: wedge)	wedge / 99.8	wedge / 94.0	wedge / 88.6	ran as 128: 89.9 / 90.1	89.9 / 31.6	89.9 / 19.4

(2026-10-03: one_channel_xfer_latency re-measured post-fix (`genesys_dw256_one_channel_xfer_latency_20261003.json`); 256-bit columns are from the re-measurement, 512-bit columns are pre-fix and were not re-run 2026-10-03.)

Three things fall out. First, the **write side needs no rebuild**: its window is outstanding x burst because the sink frees SRAM on the W handshake, not on B, so B latency costs no buffer. 64-beat bursts hold one channel at 99.5 % through 256 cycles on either build, and on the 16 KB build 128-beat bursts hold 97.9 % through 512 (the 2 % is the cost of staging a 128-beat burst before its AW; 16 outstanding at 32 beats would give the same 512-beat window without it, and `AW_MAX_OUTSTANDING` is a top-level parameter, not yet a build generic). Second, the **read side is bounded by the per-channel SRAM** and burst length cannot fix it: the read engine pre-allocates SRAM for every AR, so in-flight reads never exceed the buffer, and longer bursts only make it worse (fewer fit). The best any burst reaches at 256 cycles is 81 % with 16 KB and 44 % with 4 KB – the ratio of the buffer to the round trip – and holding one read channel at line rate through 512 cycles needs about 1024 beats of SRAM per channel, which fits the XC7K325T at 64 B beats (about 230 of 445 BRAM tiles) but is the opposite direction from the 4 KB point. Third, **AxLEN 255 was a bug, not a limit** (rapids BUG-009, fixed in v2.2). Both engines computed the burst's beat count as an 8-bit `AxLEN + 1`, which wraps to 0 at 255: the write gate passed on an empty buffer, the AW went out, `drain_ctrl` reserved nothing while the W phase pulled 256 beats, and the channel's

drain count sat at 0 for good (`CHANNEL_RESET` does not reach `sram_controller`; only a reprogram does). The read engine's mirror image allocated 0 beats and over-fetched (4103 R beats for 4096). Every later run on a poisoned channel with a burst of 120 beats or more accepted one buffer of ingress and issued no AW – the v2.1 "128-beat wedge" on the 4 KB build was that aftermath, measured after an AxLEN 255 row; on the fixed build 128-beat bursts hold 89.9 % at every latency (Table 7.7: the write window is the full buffer, so latency costs nothing, and the 10 % is the cost of staging a 128-beat burst before its AW). The engines now clamp the configured AxLEN to the buffer depth minus one (and to 254 beyond it), so a burst is never larger than the SRAM that stages it; the AxLEN 255 rows below therefore ran as 128-beat bursts (`xfer_axlen_effective` in the data file), which is why they repeat the 128 row. Two smaller mechanisms sat underneath. The sink ingress allocated SRAM only in whole 16-beat segments, so any packet that ended mid-segment (a 4-beat smoke run, say) left the last few slots of the buffer unreachable, and a burst equal to the depth – which needs every slot – waited forever; the ingress now allocates the remainder when less than a segment is free. And that partial allocation, consumed in a cycle or two, was made against a space view that lags an allocation by three cycles: the allocator over-committed, its space count wrapped, and beats were dropped on allocation cycles; each channel now settles three cycles after an allocation and never accepts a beat the FIFO cannot take. A burst up to the depth is satisfiable after any history (`test_residue_full_burst`, and the board replay of the exact run sequence that used to wedge).

Appendix: data files & reproduce

File	Contents
`perf/json/genesys_dw256_full_matrix.json`	v2.0 channel x size matrix, 256-bit / 4 KB-per-channel build (bare meters); every file of this build carries a `design` record read from the bitstream
`perf/json/genesys_dw256_obs_{A,B,C,E}.json`, `genesys_dw256_obs_E_interleave.json`	v2.0 observer campaign on that build: descriptors x channels, burst length, size x channels, latency (both schedules)
`perf/json/genesys_dw256_one_channel_xfer_latency.json`, `genesys_one_channel_xfer_latency.json`	section 7.7: one channel, burst x latency, on the 256-bit and the 512-bit build
`perf/json/genesys_dw256_fine_{4kb,16kb}_{8ch,1ch}.json`	section 7.5c: 8-cycle-step latency sweep, 4 KB build and the 16 KB (`RAPIDS_SRAM_DEPTH=512`) variant, 8 and 1 channels
`perf/json/genesys_obs_{A,B,C,E}.json`	v1.2-v1.4 observer campaign, 512-bit build: descriptors x channels, burst length, size x channels, latency
`perf/json/genesys_obs_E_interleave.json`	v1.5 latency sweep with the interleaved channel schedule (`--interleave`, Table 7.5b)
`perf/json/genesys_full_matrix.json`	channel × size matrix (v1.4, bare meters, PIPELINE = 1)
`perf/json/genesys_full_matrix_v1.1.json`	the same matrix as measured for v1.1 (PIPELINE = 0 with the pre-BUG-005 engine)
`perf/json/genesys_8ch_*.json`	earlier back-to-back runs (show the pre-fix wedge)
`perf/plots/dw256_*.png`	v2.0 figures (`flows-rapids-beats/host/report_figures.py` over the dw256 files); the unprefixed PNGs are the v1.x figures

```
# full channel x size matrix in ONE programming (CHANNEL_RESET per run):
source env_python
```

```
python3 projects/fpga-systems/Genesys2/dma-ip/rapids_beats/flows-
rapids-beats/host/run_characterization.py \
    --port /dev/ttyUSB1 --channels 8 --suite \
    --suite-channels 1,2,4,8 --suite-beats 4,16,64,256,1024,4096 --
suite-bp off \
    --results
projects/fpga-systems/Genesys2/dma-ip/rapids_beats/reports/perf/json/
genesys_full_matrix.json

# figures:
python3 projects/fpga-systems/Genesys2/dma-ip/rapids_beats/flows-
rapids-beats/host/plot_char_reports.py \
    --size
projects/fpga-systems/Genesys2/dma-ip/rapids_beats/reports/perf/json/
genesys_full_matrix.json \
    --outdir
projects/fpga-systems/Genesys2/dma-ip/rapids_beats/reports/perf/plots

# this report (DOCX + PDF, house style):
cd projects/fpga-systems/Genesys2/dma-ip/rapids_beats/reports &&
./generate_reports_pdf.sh --rev 2.2

# v2.0 (256-bit / 4 KB-per-channel design point: the Makefile's
DATA_WIDTH / SRAM_DEPTH defaults):
#   make bitstream BOARD=genesys2 USE_OBSERVERS=1
OBS_ENABLE_MON_TAPS=0 && make program BOARD=genesys2
#   the same campaign lines as below with --port auto and
genesys_dw256_* result names, plus
#   python3 $H --port auto --channels 8 --suite --suite-bp off --
suite-seeds default \
#       --suite-channels 1 --suite-beats 4096 --suite-xfer
7,15,31,63,127 --suite-delay 0,256,512 \
#       --results $J/genesys_dw256_one_channel_xfer_latency.json
# section 7.7
#   python3 flows-rapids-beats/host/report_tables.py  $J
genesys_dw256_        # every table above
#   python3 flows-rapids-beats/host/report_figures.py $J
genesys_dw256_ reports/perf/plots dw256_

# v1.2 observer campaign (USE_OBSERVERS=1 bitstream: make bitstream
BOARD=genesys2 USE_OBSERVERS=1):
H=projects/fpga-systems/Genesys2/dma-ip/rapids_beats/flows-rapids-
beats/host/run_characterization.py
J=projects/fpga-systems/Genesys2/dma-ip/rapids_beats/reports/perf/json
python3 $H --port /dev/ttyUSB0 --channels 8 --suite --suite-bp off --
suite-seeds default \
    --suite-channels 1,2,4,8 --suite-beats 1024 --suite-descs
```

```
1,2,4,8,16 --results $J/genesys_obs_A.json
python3 $H --port /dev/ttyUSB0 --channels 8 --suite --suite-bp off --
suite-seeds default \
    --suite-channels 8 --suite-beats 1024 --suite-xfer
0,1,3,7,15,31,63 --results $J/genesys_obs_B.json
python3 $H --port /dev/ttyUSB0 --channels 8 --suite --suite-bp off --
suite-seeds default \
    --suite-channels 1,2,4,8 --suite-beats 1,4,16,64,256,1024,4096 --
results $J/genesys_obs_C.json
python3 $H --port /dev/ttyUSB0 --channels 8 --suite --suite-bp off --
suite-seeds default \
    --suite-channels 8 --suite-beats 1024 --suite-delay
0,8,16,32,48,64,96,128,192,256,384,512 --results $J/genesys_obs_E.json
# v1.5: the same latency sweep with the generator round-robining the
channels per beat (Table 7.5b):
python3 $H --port auto --channels 8 --suite --suite-bp off --suite-
seeds default --interleave \
    --suite-channels 8 --suite-beats 1024 --suite-delay
0,8,16,32,48,64,96,128,192,256,384,512 --results
$J/genesys_obs_E_interleave.json
# figures (size/channel plots from C; obs_desc_matrix /
obs_xfer_knee / obs_latency_knee from A / B / E):
python3 .../host/plot_char_reports.py --size $J/genesys_obs_C.json --
outdir .../reports/perf/plots
cd projects/fpga-systems/Genesys2/dma-ip/rapids_beats/reports &&
./generate_reports_pdf.sh --rev 1.5 --only perf
```

Genesys 2 host link: JTAG on the FT2232 (`200300B818A0`), UART on the separate FT232R (`AU05X8RM`, `/dev/ttyUSB1`); both must be connected at once, and the board must not be power-cycled between program and run.